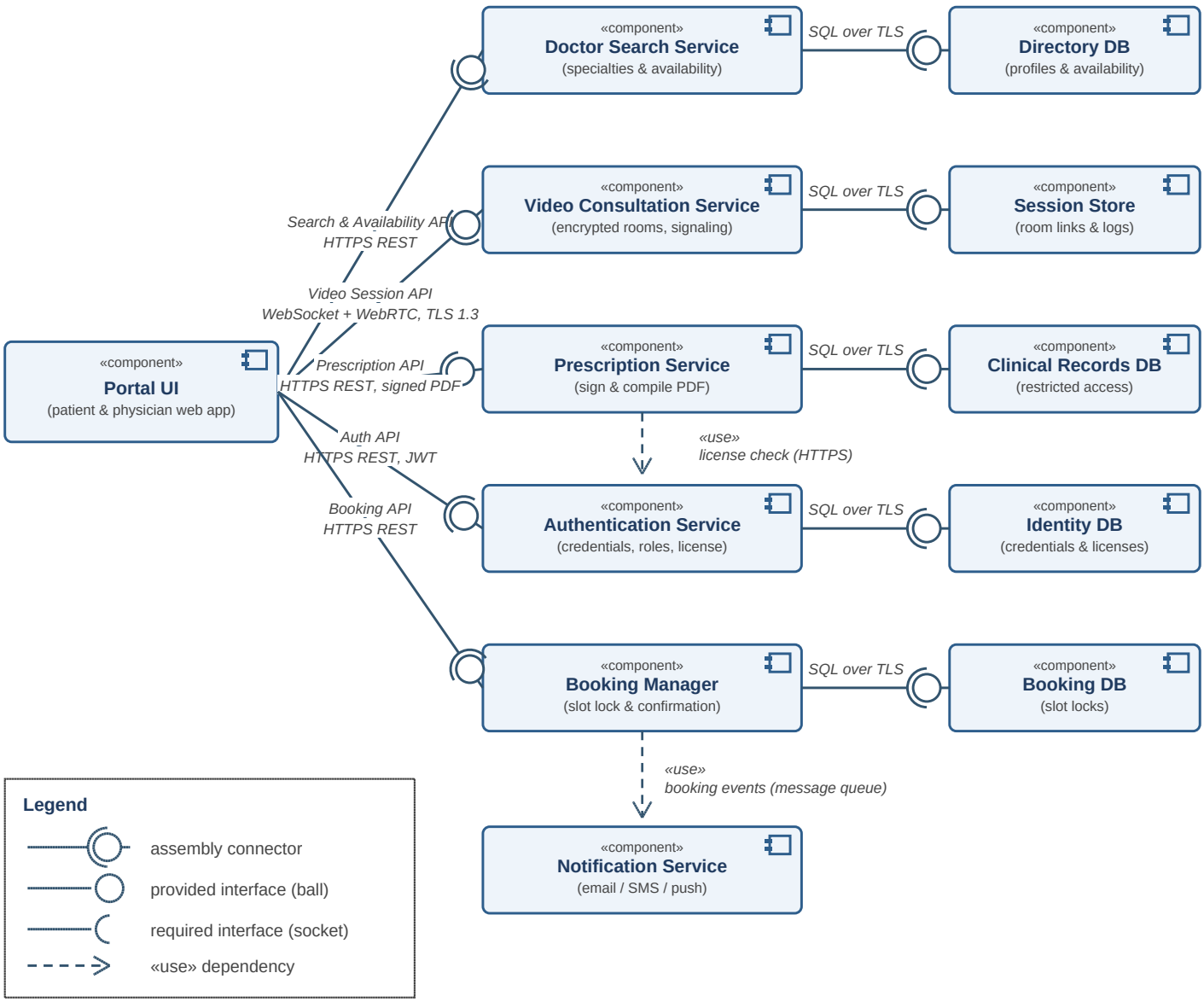
SE Lab 3: Component Modelling & Architectural Pattern

Lakshya Budhauliya

Scenario: Telemedicine Slot Booking & Prescription Portal. Remote patients search doctors by specialty, book video consultation slots, join encrypted tele-consultation sessions and download digitally signed prescriptions. Physicians manage availability, conduct consultations and author signed prescriptions.

Part 1 - UML Component Diagram

Telemedicine Slot Booking & Prescription Portal - UML Component Diagram



Interfaces

#	Interface	Provided by	Required by	Technology / Protocol
1	Search & Availability API	Doctor Search Service	Portal UI	HTTPS REST
2	Booking API	Booking Manager	Portal UI	HTTPS REST
3	Video Session API	Video Consultation Service	Portal UI	WebSocket signaling + WebRTC, TLS 1.3
4	Prescription API	Prescription Service	Portal UI	HTTPS REST, signed PDF download
5	Auth API	Authentication Service	Portal UI	HTTPS REST, JWT tokens
6	Persistence (5 stores)	Each service's own data store	Its owning service only	SQL over TLS; clinical store has restricted credentials
7	Booking events («use»)	Notification Service	Booking Manager	Message queue
8	License validation («use»)	Authentication Service	Prescription Service	HTTPS REST

Part 2 - Written Justification

We chose the Microservices architecture for the Telemedicine Slot Booking & Prescription Portal.

The system is split into small services that each own one job: an Authentication Service for login and role checks, a Doctor Search Service for browsing specialties and real-time availability, a Booking Manager for reserving slots, a Video Consultation Service for the encrypted consultation rooms, a Prescription Service for digitally signed prescriptions, and a Notification Service for confirmations and room links. Each service is deployed, scaled and restarted on its own, owns its own data store, and talks to the other services only through fixed APIs.

Why it fits this system

1. The parts of this scenario have very different workloads. A video consultation is a long, heavy real-time session, while doctor search and slot booking are short, light requests. With microservices the video service can be given more instances during busy consultation hours on its own, without also scaling search or booking.

2. Our NFR-002 asks for 99.9% uptime with automatic failover for the video signaling service while the booking subsystem keeps running. Because each service is deployed separately, a crash or an update in one service, say doctor search, is much less likely to take booking or video down with it, and the video service can fail over to a standby instance on its own. This reduces the blast radius of a failure rather than guaranteeing isolation, since a service can still be affected when a dependency it calls is down. One combined deployment would restart and fail as a single unit, which makes that target much harder to reach.

Security advantage

Each service owns its own data store with its own credentials. Patient clinical notes and signed prescriptions live only in the Clinical Records DB behind the Prescription Service, and the other services, including doctor search, have no access to that store at all. So if a lower-sensitivity part like doctor search is compromised, the exposure is limited to public directory data rather than prescriptions or consultation details. This limits how far a breach can spread. It reduces risk rather than removing it, and healthcare-data compliance still depends on how each service is actually implemented.

Performance benefit

Each service scales on its own, so capacity goes where the load actually is: more video signaling instances at peak consultation hours and more replicas of the doctor directory when many patients are browsing, while the booking path stays small and fast. Because the directory is its own store, search reads can be cached and replicated without touching booking or clinical data. Calls between services do cross the network, which costs some latency, but for this mix of light requests and heavy video sessions the independent scaling is worth the trade-off.